

Chapter 29: LangGraph Error Handling, Retry Architectures, Compensation Patterns, Circuit Breakers, and Resilient Agent Systems

Part III: LangGraph Deep Dive

Book: Agentic AI & RAG Systems Handbook

Version: 2.0 Expanded Edition

Chapter ID: 29

File Name: chapter-29-langgraph-error-handling-and-resilience.md

Chapter Overview

Most AI demonstrations assume:

1 Everything Works

Production environments assume:

1 Everything Eventually Fails

Failures occur everywhere:

- LLM APIs fail
- Tool calls time out
- Vector databases become unavailable
- Network requests fail
- Agents return malformed outputs
- Human approvals never arrive
- External systems reject requests
- State becomes inconsistent

Enterprise-grade agent platforms are not defined by how they behave during success.

They are defined by how they behave during failure.

This chapter focuses on building:

1 Reliable
2
3 Fault-Tolerant
4

5	Recoverable
6	
7	Resilient

LangGraph systems.

Learning Objectives

After completing this chapter you should be able to:

- Design fault-tolerant LangGraph workflows
 - Implement retry architectures
 - Build compensation workflows
 - Design circuit breakers
 - Handle external dependency failures
 - Implement graceful degradation
 - Build resilient multi-agent systems
 - Design enterprise recovery strategies
 - Explain resilience patterns in architect interviews
-

Prerequisites

Required:

- Chapter 22: Checkpointing and Durable Execution
 - Chapter 23: Interrupts and Resume Semantics
 - Chapter 24: Multi-Agent Architectures
 - Chapter 28: Observability and Monitoring
-

1. The Reality of Failure

Every production system eventually encounters:

1	Service Outage
2	
3	Dependency Failure
4	
5	Network Loss

- | | |
|---|----------------|
| 6 | |
| 7 | Corrupted Data |
| 8 | |
| 9 | Timeouts |

A resilient agent platform assumes failure is normal.

Reliability Principle

Bad Architecture:

- | | |
|---|---------|
| 1 | Failure |
| 2 | ↓ |
| 3 | Crash |

Good Architecture:

- | | |
|---|----------|
| 1 | Failure |
| 2 | ↓ |
| 3 | Recover |
| 4 | ↓ |
| 5 | Continue |

2. Failure Taxonomy

Understanding failure types is critical.

Transient Failures

Temporary failures.

Examples:

- | | |
|---|-----------------|
| 1 | Network Timeout |
| 2 | |
| 3 | Rate Limiting |
| 4 | |
| 5 | API Overload |

Usually recoverable.

Persistent Failures

Long-lasting failures.

Examples:

- | | |
|---|------------------------|
| 1 | Database Down |
| 2 | |
| 3 | Service Outage |
| 4 | |
| 5 | Authentication Failure |

Require escalation.

Business Failures

Workflow-specific failures.

Examples:

- | | |
|---|-------------------|
| 1 | Approval Rejected |
| 2 | |
| 3 | Policy Denied |
| 4 | |
| 5 | Validation Failed |

Expected outcomes.

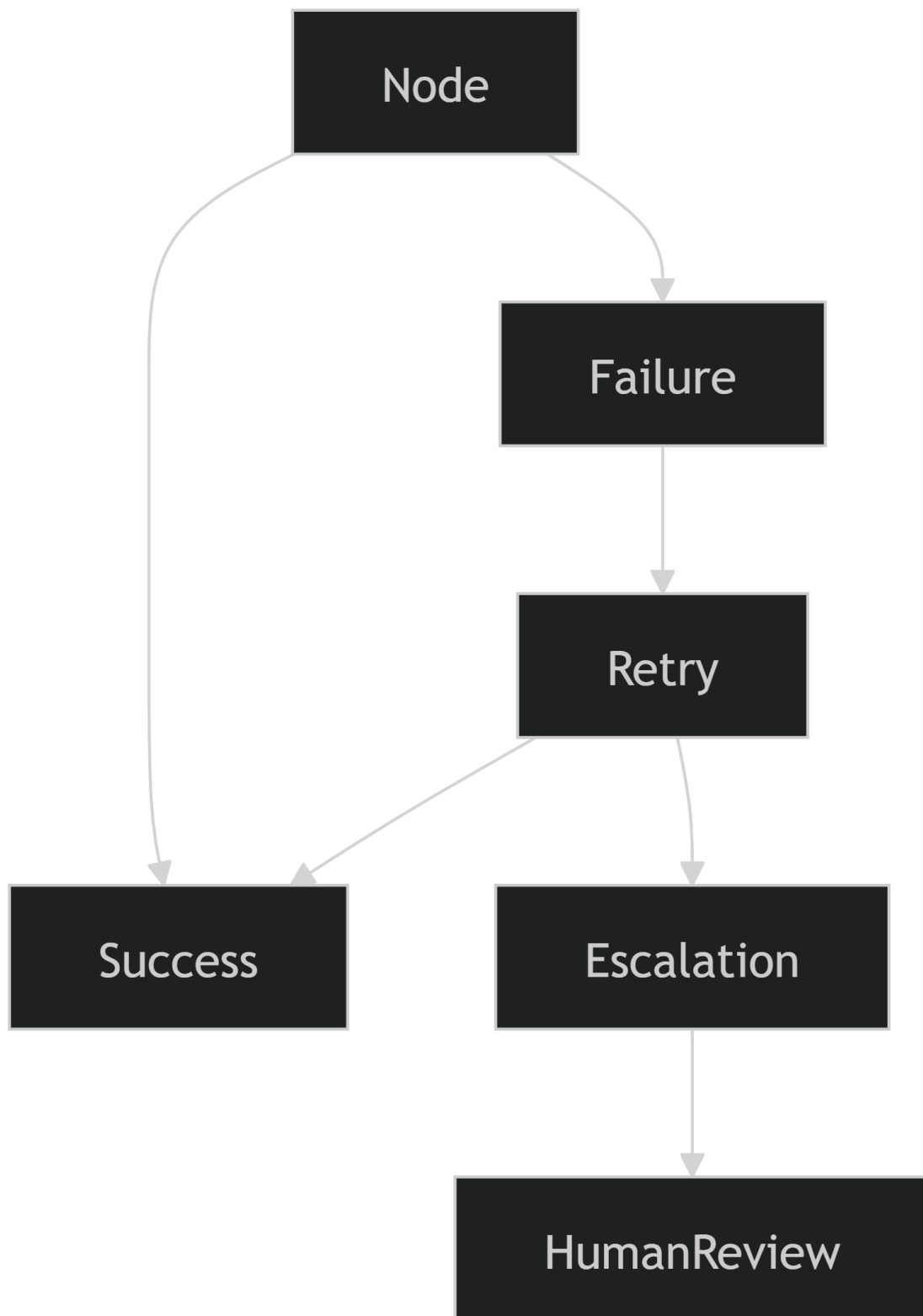
System Failures

Infrastructure failures.

Examples:

- | | |
|---|-------------------|
| 1 | Container Crash |
| 2 | |
| 3 | Node Failure |
| 4 | |
| 5 | Memory Exhaustion |
-

3. Error Handling Architecture



4. Error Handling Principles

Principle 1

Assume every dependency fails.

Principle 2

Never lose state.

Principle 3

Retry intelligently.

Principle 4

Fail safely.

Principle 5

Log every failure.

Principle 6

Always design recovery paths.

5. Node-Level Error Handling

Every node should anticipate:

- | | |
|---|--------------------|
| 1 | Tool Failure |
| 2 | |
| 3 | API Failure |
| 4 | |
| 5 | Validation Failure |
| 6 | |
| 7 | Timeout |

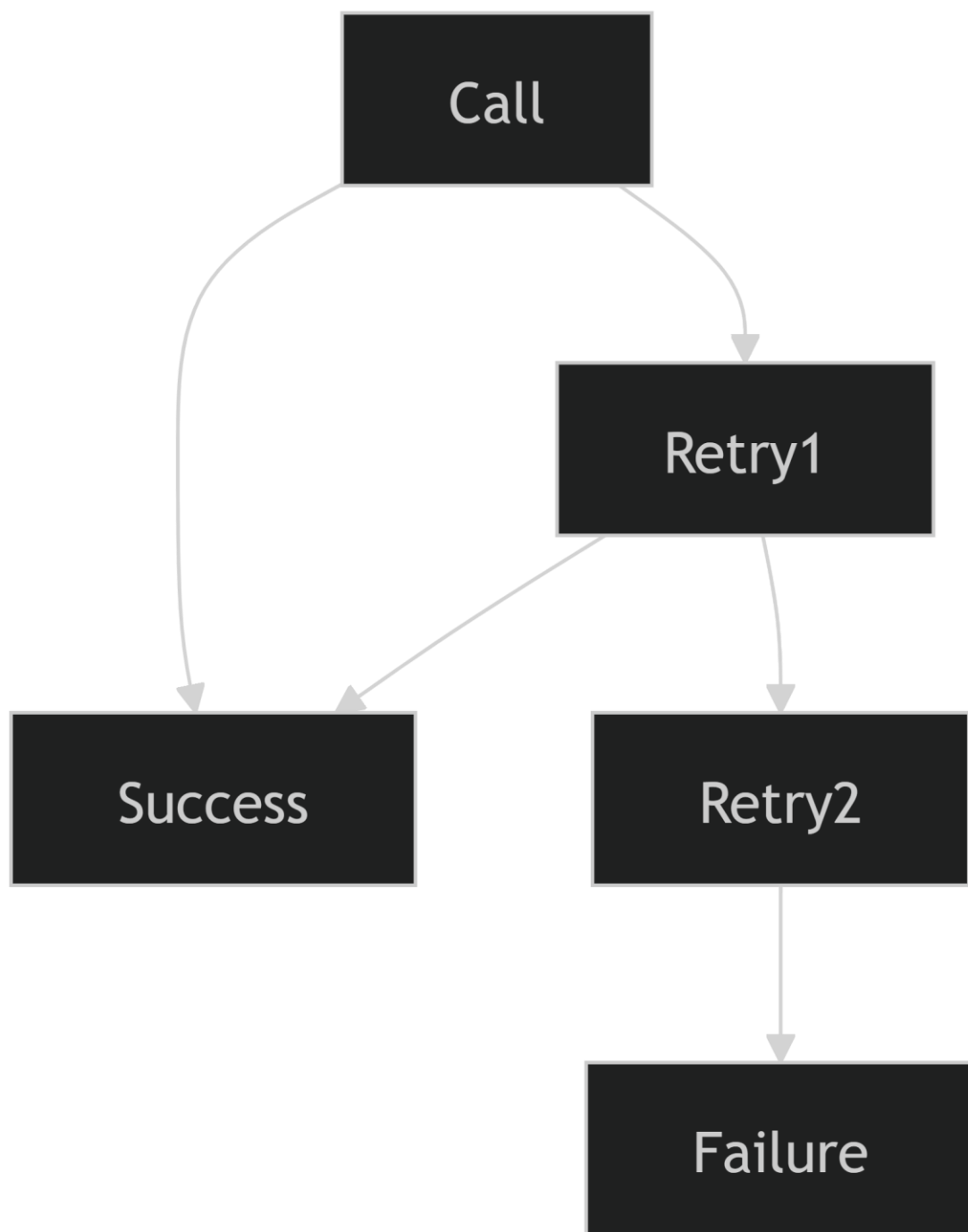
Example

```
1  def search_node(state):
2
3      try:
4
5          results = search(
6              state["query"]
7          )
8
9          return {
10             "results": results
11         }
12
13     except Exception as ex:
14
15         return {
16             "error": str(ex)
17         }
```

6. Retry Pattern

The most common resilience strategy.

Architecture:



7. Retry Strategies

Immediate Retry

1 Fail

- | | |
|---|-------|
| 2 | ↓ |
| 3 | Retry |

Useful for:

- | | |
|---|------------------|
| 1 | Network Glitches |
|---|------------------|
-

Fixed Delay Retry

- | | |
|---|----------------|
| 1 | Fail |
| 2 | ↓ |
| 3 | Wait 5 Seconds |
| 4 | ↓ |
| 5 | Retry |
-

Exponential Backoff

- | | |
|---|-----------|
| 1 | 1 Second |
| 2 | |
| 3 | 2 Seconds |
| 4 | |
| 5 | 4 Seconds |
| 6 | |
| 7 | 8 Seconds |

Preferred enterprise pattern.

Jitter

Add randomness:

- | | |
|---|-------------|
| 1 | 4.3 Seconds |
| 2 | |
| 3 | 8.1 Seconds |

Avoids retry storms.

8. Retry Classification

Not all failures should retry.

Safe To Retry

- | | |
|---|---------------------------|
| 1 | Timeout |
| 2 | |
| 3 | Temporary Network Failure |
| 4 | |
| 5 | Rate Limiting |

Do Not Retry

- | | |
|---|------------------------|
| 1 | Authentication Failure |
| 2 | |
| 3 | Permission Denied |
| 4 | |
| 5 | Invalid Request |

9. Fallback Pattern

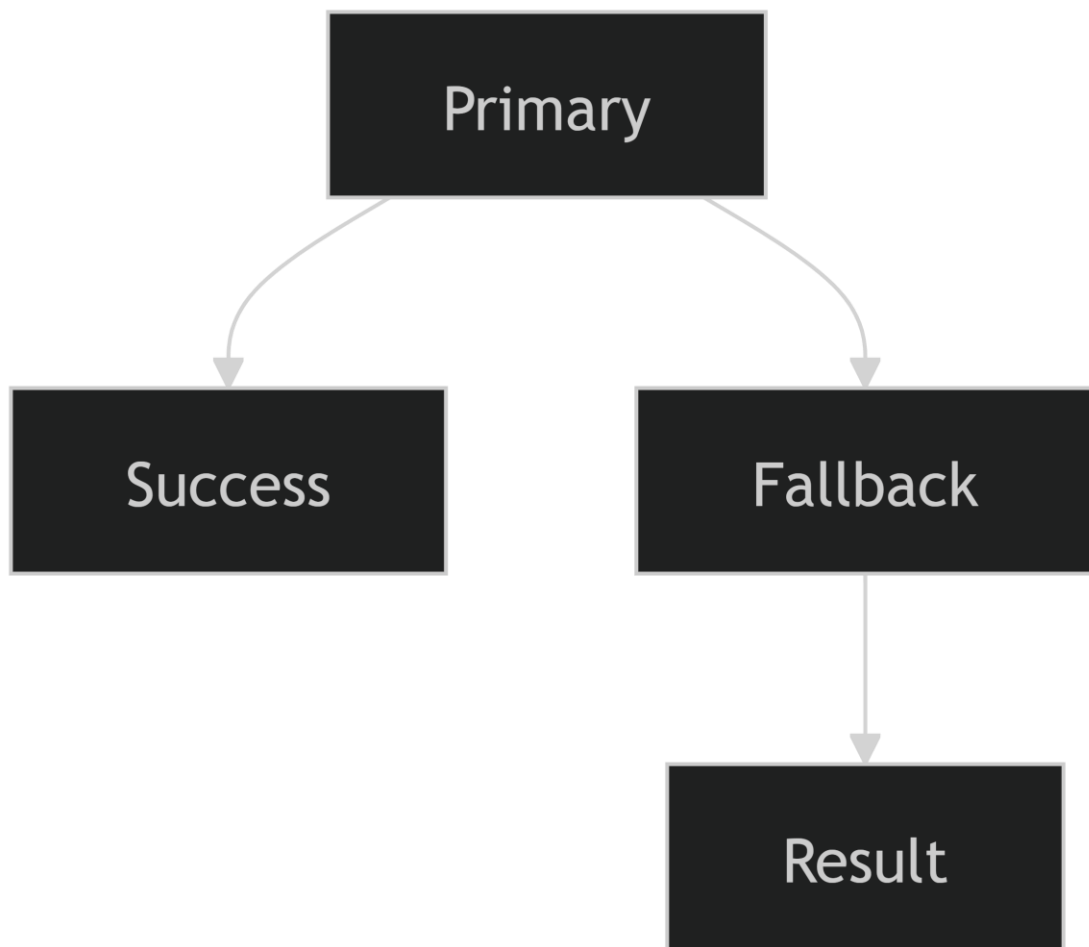
If primary path fails:

Use backup path.

Example

- | | |
|---|---------------|
| 1 | Vector Search |
| 2 | ↓ |
| 3 | Failed |
| 4 | ↓ |
| 5 | BM25 Search |

Architecture:



10. Graceful Degradation

Do something useful when ideal behavior is impossible.

Example

Instead Of:

1 Failure

Return:

1 Limited Results

Example

1 Knowledge Search Failed

Use:

1 Cached Results

11. Circuit Breaker Pattern

One of the most important enterprise patterns.

Problem

Repeated retries overwhelm failing services.

1 Service Down
2
3 1000 Requests
4
5 1000 Failures

Solution

Circuit Breaker:

1 Failure Threshold Reached
2 ↓
3 Stop Calls
4 ↓
5 Wait
6 ↓
7 Test Recovery

12. Circuit Breaker States

Closed

Normal operation.

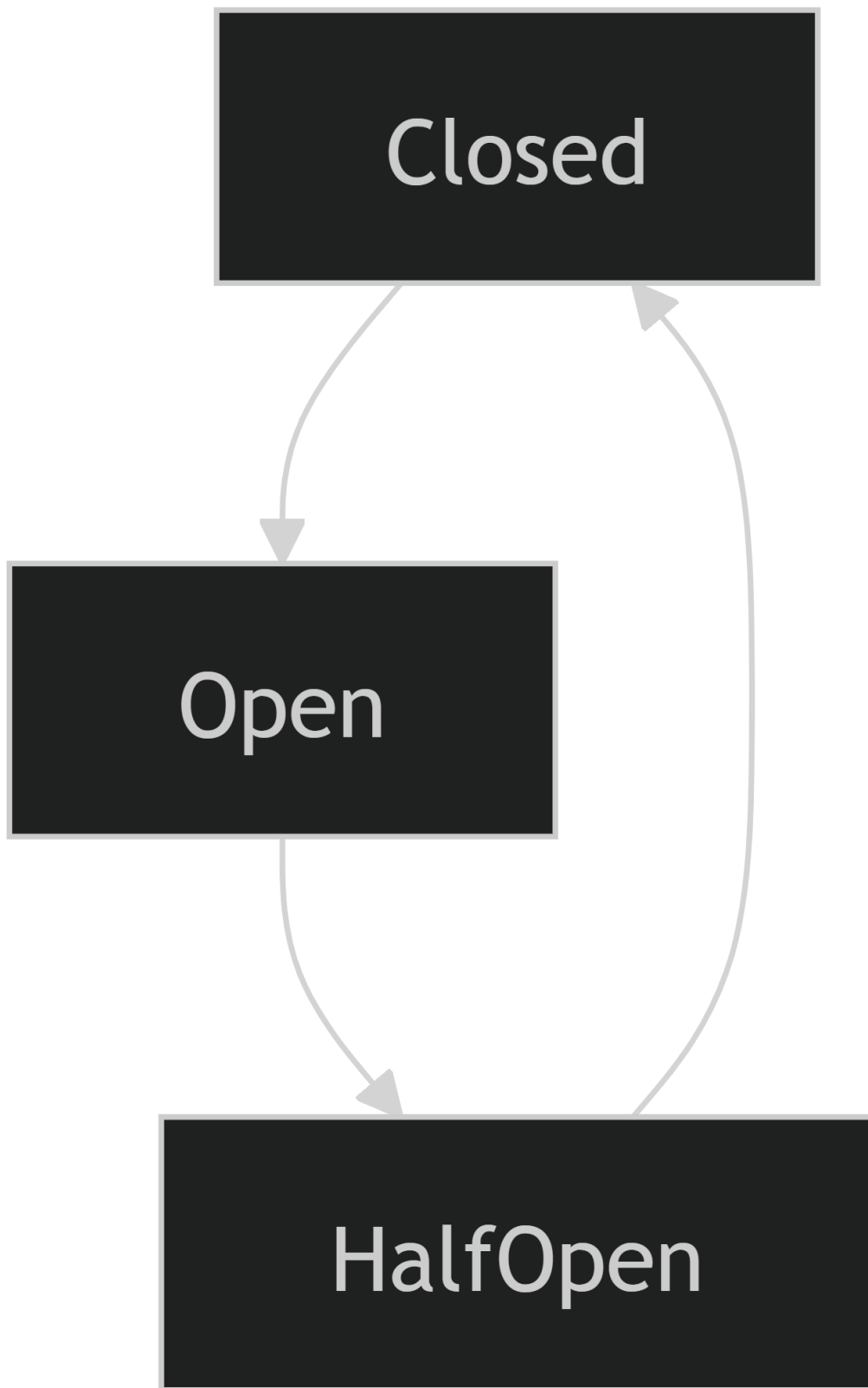
Open

Traffic blocked.

Half-Open

Testing recovery.

Architecture



13. LangGraph Circuit Breakers

Implementation Concept:

```
1  if service_health == "down":  
2  
3      return "fallback"
```

Routing occurs before failure propagation.

14. Compensation Pattern

Critical for workflows with side effects.

Example

Workflow:

```
1  Create Account  
2  
3  Assign License  
4  
5  Create Ticket
```

Failure:

```
1  License Assignment Fails
```

Compensation:

```
1  Delete Account
```

15. Compensation Architecture

ActionA



ActionB



Failure



UndoA

Concept

Not rollback.

Compensate.

16. Saga Pattern

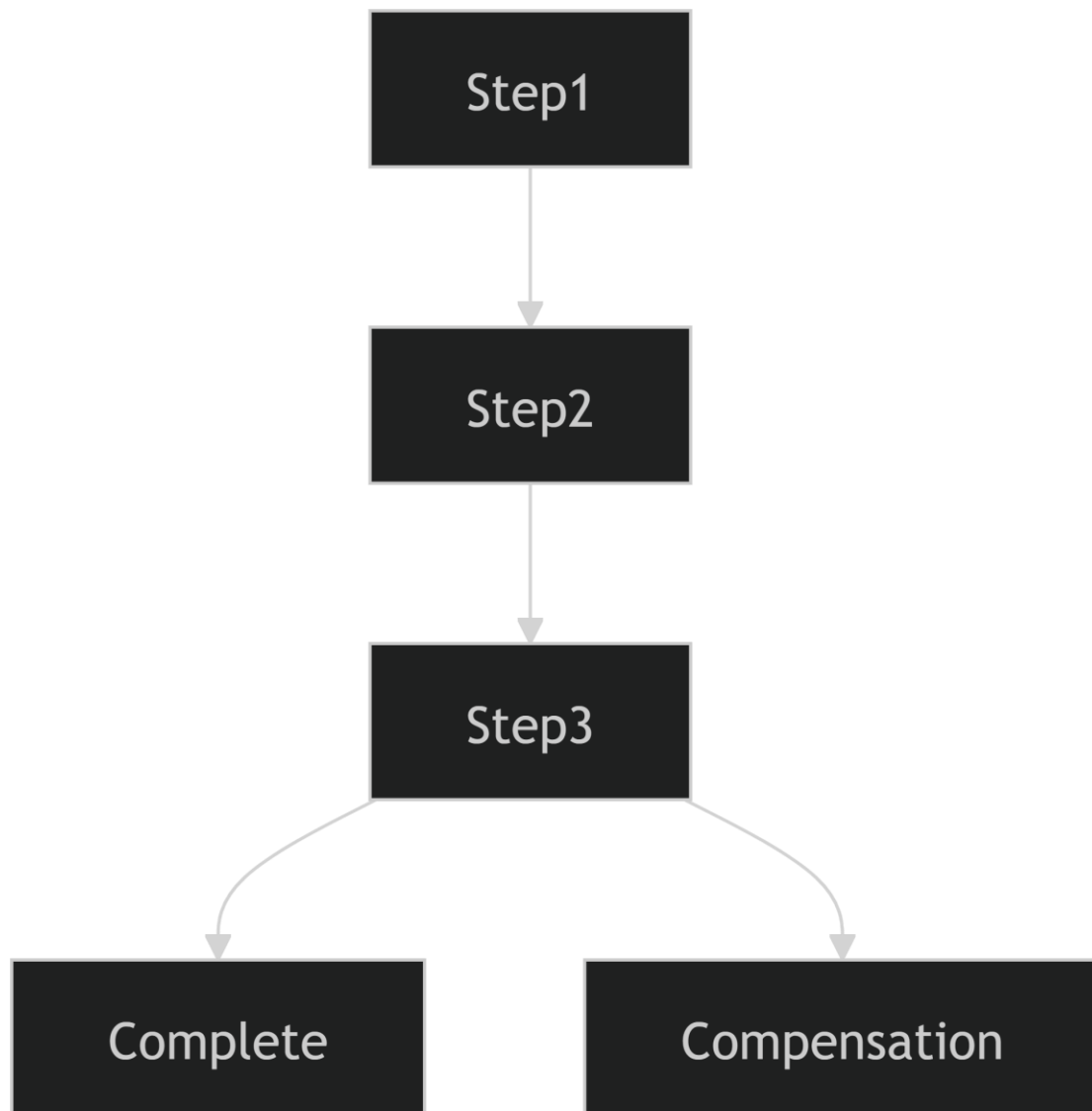
Enterprise transactions frequently span multiple systems.

Example:

1	CRM
2	
3	Billing
4	
5	Identity
6	
7	Support

No global transaction exists.

Architecture



17. Timeout Pattern

Every workflow should define limits.

Bad

1 Wait Forever

Good

```
1  timeout:  
2    30s
```

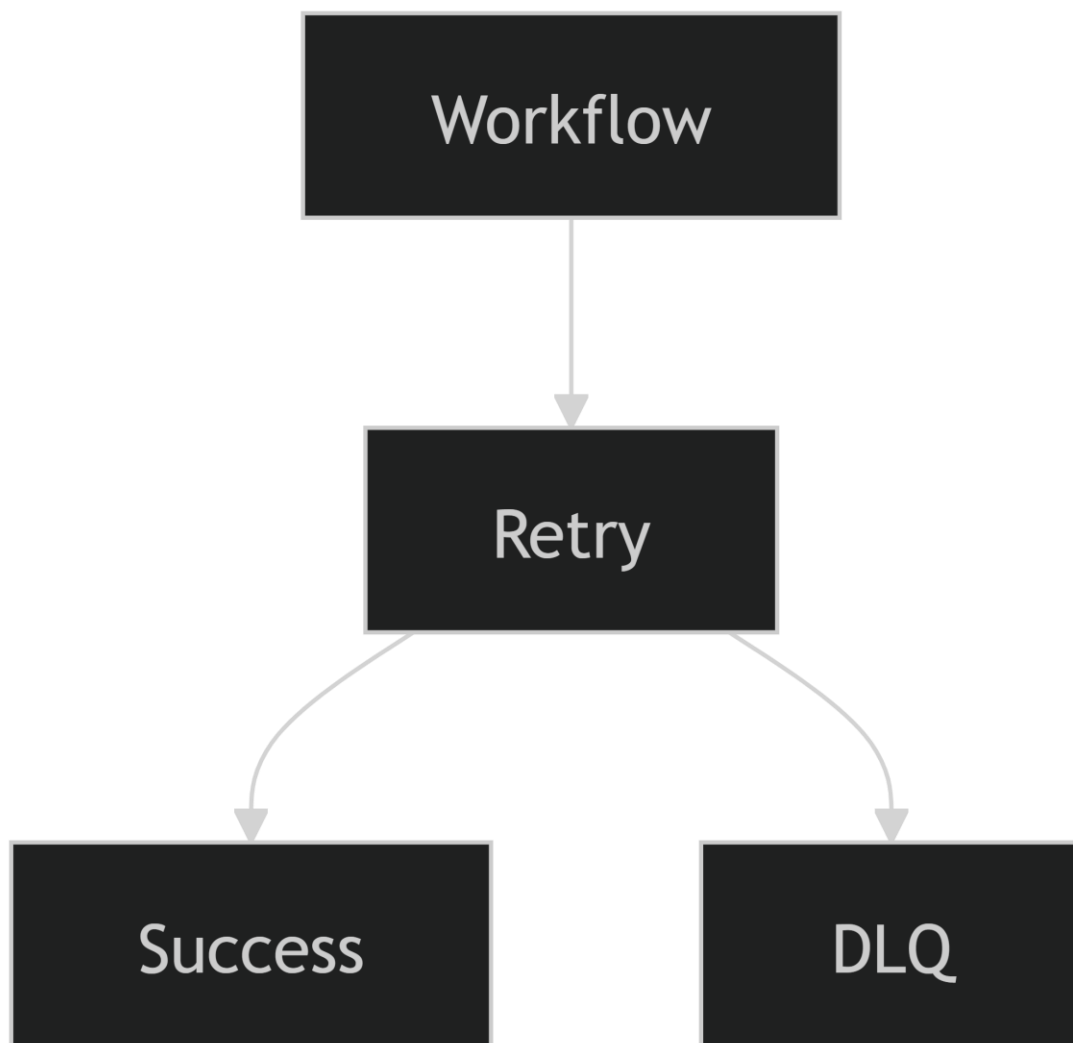
Examples

```
1  Tool Calls  
2  
3  Approvals  
4  
5  External APIs
```

18. Dead Letter Queue (DLQ)

Failed workflows require safe handling.

Architecture:



Use cases:

- | | |
|---|----------------------|
| 1 | Repeated Failures |
| 2 | |
| 3 | Manual Investigation |
| 4 | |
| 5 | Forensics |

19. Human Escalation Pattern

Some failures cannot be automated.

Workflow

```
1  Failure
2
3  ↓
4
5  Retry
6
7  ↓
8
9  Escalate
10
11 ↓
12
13 Human Review
```

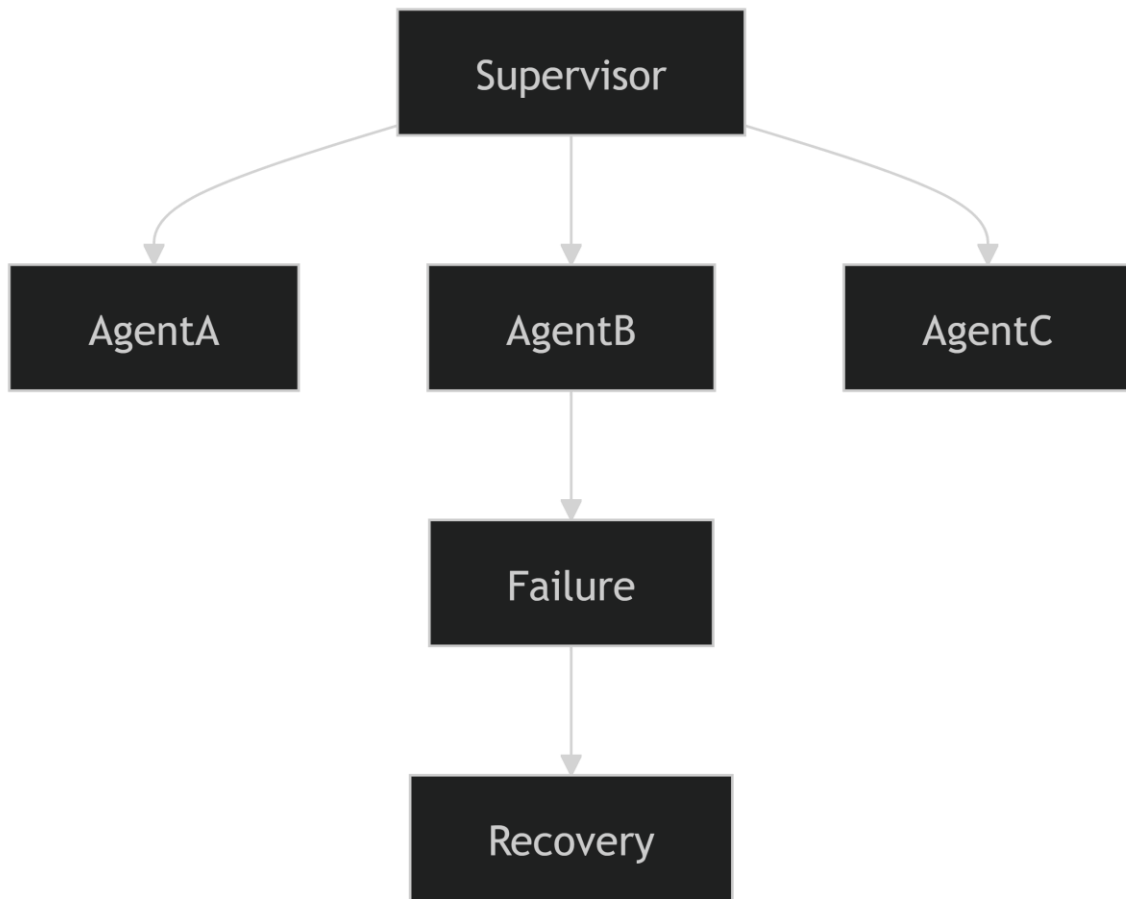
Examples:

```
1  Policy Violation
2
3  Ambiguous Data
4
5  Fraud Detection
```

20. Multi-Agent Resilience

Agent failures should not destroy the entire workflow.

Architecture:



Preferred behavior:

1 Partial Completion

instead of:

1 Global Failure

21. Checkpoint-Based Recovery

Leverage Chapter 22 concepts.

Workflow:

1 Checkpoint A
2
3 Checkpoint B
4

- | | |
|---|---------------|
| 5 | Failure |
| 6 | |
| 7 | Resume From B |

Benefits:

- | | |
|---|-------------------|
| 1 | Reduced Work Loss |
|---|-------------------|

22. Interrupt Recovery

Interrupts introduce unique challenges.

Examples:

- | | |
|---|---------------------|
| 1 | Approval Timing Out |
| 2 | |
| 3 | Approval Rejected |
| 4 | |
| 5 | User Never Responds |

Solutions:

- | | |
|---|-------------|
| 1 | Reminder |
| 2 | |
| 3 | Escalation |
| 4 | |
| 5 | Auto-Reject |

23. Retrieval Failure Recovery

Common RAG failures:

- | | |
|---|-------------------|
| 1 | No Results |
| 2 | |
| 3 | Low Relevance |
| 4 | |
| 5 | Vector Store Down |

Recovery:

- | | |
|---|-----------------|
| 1 | Rewrite Query |
| 2 | |
| 3 | Fallback Search |

4

5 Alternative Retriever

24. LLM Failure Recovery

Examples:

1 Malformed JSON

2

3 Empty Response

4

5 Rate Limiting

Recovery:

1 Retry

2

3 Schema Repair

4

5 Fallback Model

25. Structured Output Recovery

Example:

Expected:

1 {

2 "severity":"high"

3 }

Received:

1 Severity appears high.

Recovery:

1 Repair Output

2

3 Retry Generation

4

5 Validation Loop

26. Reliability SLOs

Define targets.

Examples:

- | | |
|---|--------------------|
| 1 | 99.9% Availability |
| 2 | |
| 3 | 99.5% Success Rate |
| 4 | |
| 5 | < 2% Retry Rate |
-

Monitor continuously.

27. Error Budget Concept

From SRE practices.

Example:

- | | |
|---|------------------|
| 1 | Target: |
| 2 | 99.9% |
| 3 | |
| 4 | Allowed Failure: |
| 5 | 0.1% |

Exceeding target triggers:

- | | |
|---|--------------------|
| 1 | Stabilization Work |
|---|--------------------|
-

28. Resilience Telemetry

Track:

Error Rate

Retry Count

Timeouts

Circuit Breaker Opens

Recovery Success

Escalation Rate

Compensation Executions

29. Security Failures

Security errors deserve special treatment.

Examples:

- | | |
|---|-------------------------|
| 1 | Unauthorized Tool Usage |
| 2 | |
| 3 | Prompt Injection |
| 4 | |
| 5 | Policy Violations |

Response:

- | | |
|---|----------|
| 1 | Block |
| 2 | |
| 3 | Audit |
| 4 | |
| 5 | Escalate |

Never retry blindly.

30. Chaos Engineering for Agents

Advanced enterprise practice.

Inject failures intentionally.

Examples:

- | | |
|---|------------------|
| 1 | Tool Failure |
| 2 | |
| 3 | Vector DB Outage |
| 4 | |
| 5 | Network Timeout |

Observe behavior.

Goal:

- | | |
|---|---------------------|
| 1 | Validate Resilience |
|---|---------------------|
-

31. Enterprise Runbooks

Every production platform should maintain:

Tool Failure Runbook

Timeout Runbook

Retry Storm Runbook

Circuit Breaker Runbook

LLM Failure Runbook

Retrieval Failure Runbook

32. Failure Modes

Failure 1

Infinite Retries

Failure 2

Retry Storms

Failure 3

State Corruption

Failure 4

Compensation Failure

Failure 5

Escalation Loops

33. Common Mistakes

Mistake 1

Retry everything.

Mistake 2

No timeout policy.

Mistake 3

Ignoring compensation.

Mistake 4

No circuit breakers.

Mistake 5

No DLQ.

34. Anti-Patterns

Crash On First Error

Unacceptable.

Infinite Recovery Loops

Consumes resources endlessly.

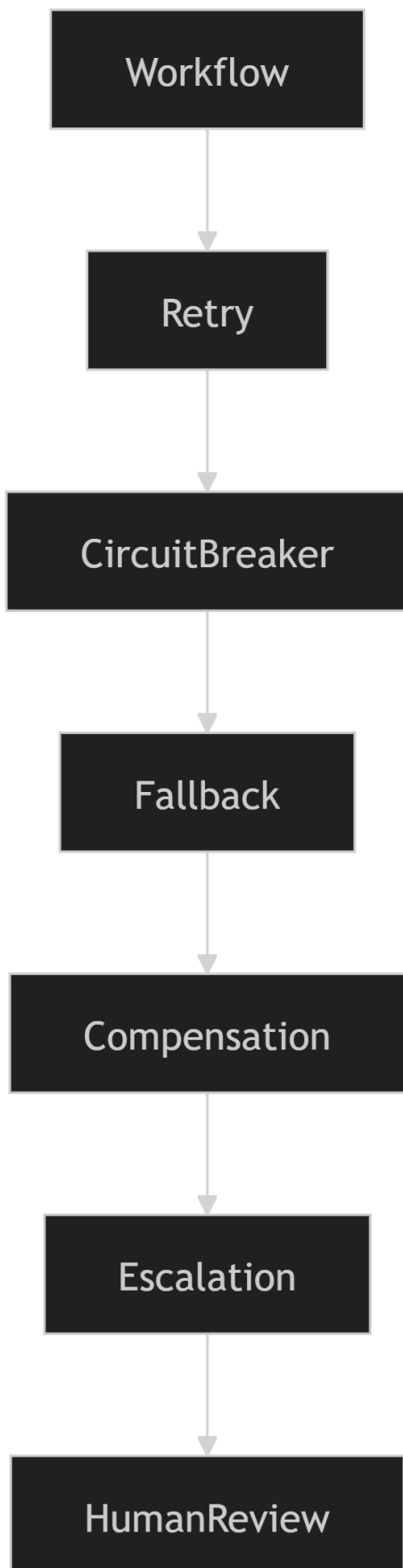
Hidden Failures

Errors silently ignored.

No Human Escalation

Recovery impossible.

35. Enterprise Reference Architecture



36. Trade-Off Analysis

Benefit	Cost
Higher reliability	More complexity
Better recovery	Additional infrastructure
Reduced downtime	More testing
Improved governance	Operational overhead

37. Architecture Decision Matrix

Pattern	Importance
Retries	Critical
Timeouts	Critical
Circuit Breakers	Critical
Compensation	High
DLQ	High
Human Escalation	High
Chaos Testing	Advanced

38. Real-World Use Cases

Enterprise Copilots

Fallback retrieval and model strategies.

IT Operations

Automated recovery workflows.

Compliance Systems

Approval timeout handling.

Financial Systems

Compensation transactions.

Digital Workforce Platforms

Workflow-level fault isolation.

39. Interview Questions

Q1

Why is error handling different in agent systems?

Q2

What failures should be retried?

Q3

What is a circuit breaker?

Q4

What is a compensation pattern?

Q5

What is the difference between rollback and compensation?

Q6

How would you design a resilient LangGraph workflow?

Q7

Why are dead letter queues important?

Q8

How do retries, checkpoints, and interrupts work together?

40. Interview Answers

Q1

Agent systems involve LLMs, retrieval, tools, routing, and external systems, creating more failure points than traditional applications.

Q2

Transient failures such as timeouts, rate limits, and network interruptions.

Q3

A protective mechanism that stops calling failing services after a threshold is reached.

Q4

A workflow that reverses or mitigates completed actions after subsequent failures occur.

Q5

Rollback restores transactional state; compensation performs corrective actions after business operations complete.

Q6

Using retries, timeouts, checkpointing, circuit breakers, fallback paths, DLQs, observability, and escalation mechanisms.

Q7

They prevent permanent loss of failed workflows and enable investigation and recovery.

Q8

Retries handle transient issues, checkpoints preserve progress, and interrupts support long-running recovery and approvals.

41. Architecture Review Checklist

Resilience Readiness Assessment

- Retry policies defined
 - Timeout policies implemented
 - Circuit breakers configured
 - Compensation workflows documented
 - DLQ architecture implemented
 - Escalation paths defined
 - Recovery testing completed
 - Chaos testing performed
 - Observability configured
 - Runbooks maintained
-

42. Practical Exercises

1. Design a circuit-breaker architecture for a RAG platform.
2. Implement retry and exponential backoff strategies.
3. Build a compensation workflow for multi-step approvals.

4. Create a dead-letter queue architecture.
 5. Design a fault-tolerant supervisor-agent system.
-

43. Key Takeaways

- Production systems are defined by failure handling, not success paths.
 - Retries must be selective and intelligent.
 - Circuit breakers protect downstream systems from cascading failures.
 - Compensation patterns are essential for workflows with side effects.
 - Checkpointing dramatically reduces recovery cost.
 - Graceful degradation improves user experience during outages.
 - DLQs prevent silent workflow loss.
 - Observability and resilience are inseparable.
 - Chaos testing validates real-world readiness.
 - Enterprise-grade LangGraph systems require resilience architectures from day one.
-

44. References

- LangGraph Documentation
 - Temporal Failure Handling Patterns
 - Martin Fowler Circuit Breaker Pattern
 - Google Site Reliability Engineering Handbook
 - AWS Resilience Engineering Guides
 - Azure Well-Architected Reliability Pillar
 - NIST Resilient Systems Guidance
 - Enterprise Integration Patterns
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 30 – LangGraph Performance Engineering, Scalability Patterns, Optimization Strategies, Cost Management, and Enterprise Capacity Planning